BreadGuard AI

AI-Based Bread Spoilage Detection Using Deep Learning

Student: Hafiz Muhammad Sufiyan Khan
Class: BSCS7A
Department: Computer Science
Instructor: Dr. Munir

Abstract

BreadGuard AI is an academic deep-learning web application for classifying photographs of bread into Fresh (no visible spoilage detected) and Spoiled (visible spoilage patterns detected). The project compares a custom convolutional neural network (CNN V4) with a MobileNetV2 transfer-learning model. On a held-out set of 73 images, the models achieved 89.04% and 94.52% test accuracy, respectively. The system is intended for visual screening, not food-safety certification.

1. Problem Statement and Objectives

Bread may develop visible mold and other spoilage patterns during storage. Manual visual assessment can vary with image quality, lighting, and observer judgment. This project investigates whether image classification can assist in identifying visible spoilage. Objectives are to curate a two-class dataset, compare CNN and MobileNetV2 models, evaluate held-out predictions, and provide an accessible browser interface.

2. Dataset and Preparation

Images were manually curated from publicly accessible online image-search results. Exact duplicate files were removed using SHA-256 hashing. The resulting dataset contains 472 unique images: 374 Fresh and 98 Spoiled. Images are processed as RGB and resized to 224 × 224 pixels. The class distribution is imbalanced, so spoiled-class recall is reported alongside accuracy. Images were sourced online; public redistribution rights have not been established.

Split	Fresh	Spoiled	Total
Training	261	68	329
Validation	56	14	70
Testing	57	16	73
Total	374	98	472

3. Model Architectures

Custom CNN V4: a convolutional network trained from scratch with convolutional and max-pooling layers, image augmentation, dense layers, dropout, and a sigmoid binary-classification output. Saved artifact: bread_scratch_cnn.keras. MobileNetV2: an ImageNet-pretrained MobileNetV2 frozen feature extractor with a

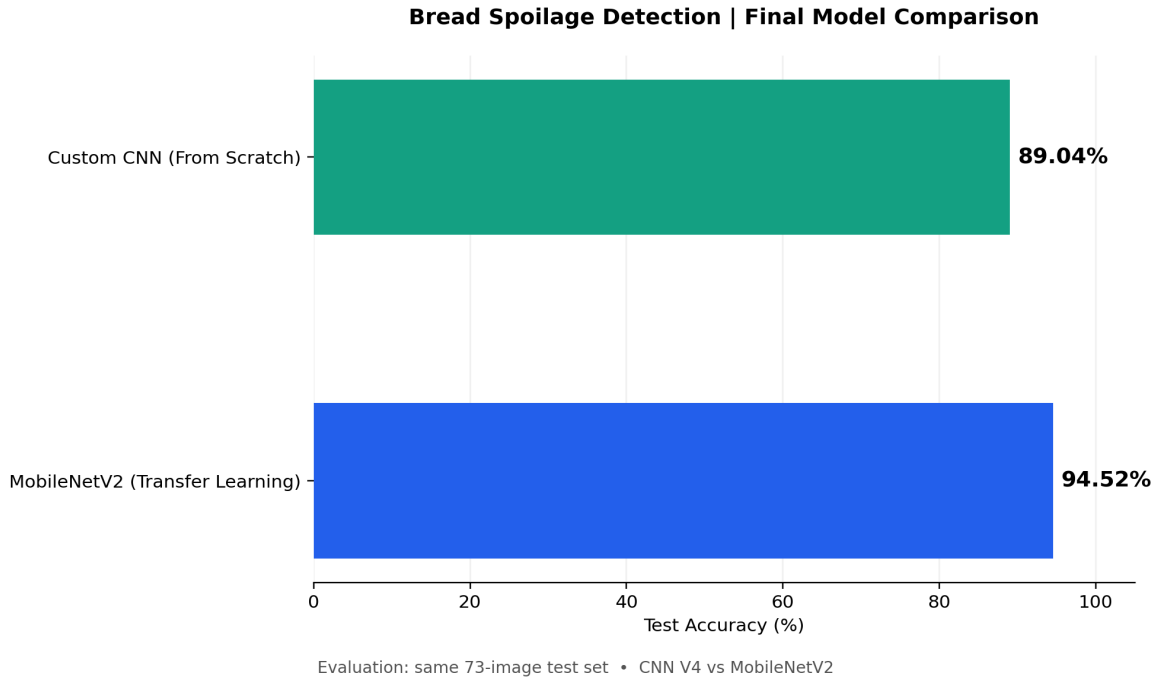
custom classification head. Saved artifact: best_mobilenetv2.keras. Both saved models include their required preprocessing; inference receives RGB image values without extra rescaling.

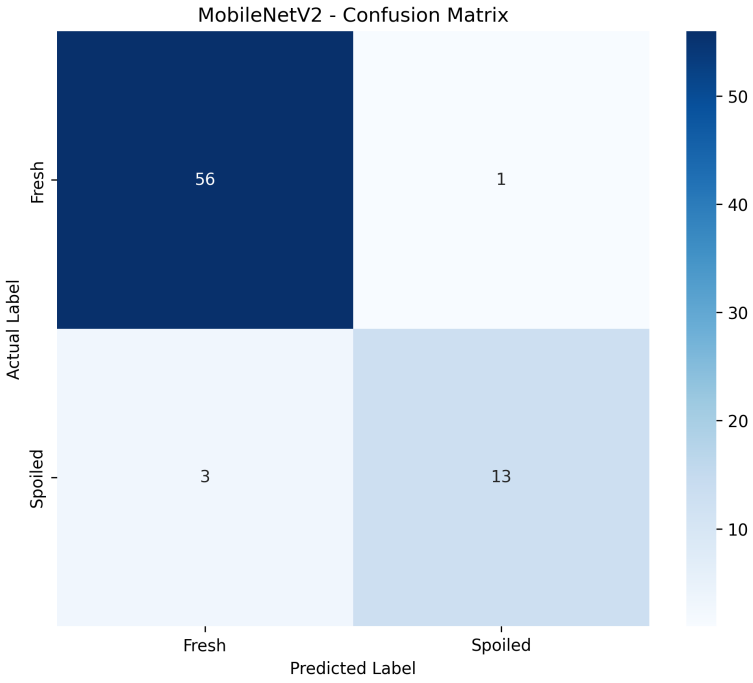
4. Evaluation and Results

Models were evaluated on the same held-out test set (57 Fresh, 16 Spoiled). The custom CNN correctly classified 53 Fresh and 12 Spoiled images, with 4 Fresh false positives and 4 Spoiled false negatives. MobileNetV2 correctly classified 56 Fresh and 13 Spoiled images, with 1 Fresh false positive and 3 Spoiled false negatives. MobileNetV2 performed better on this test set, but the sample is small.

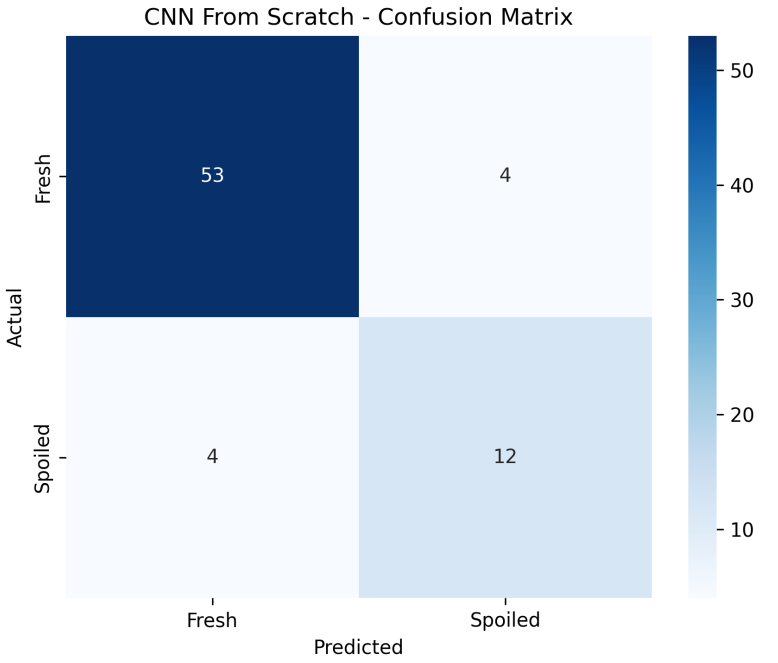
Metric	Custom CNN V4	MobileNetV2
Training accuracy	92.40%	97.57%
Validation accuracy	90.00%	97.14%
Test accuracy	89.04%	94.52%
Spoiled recall	75.00%	81.25%

Final model comparison: Custom CNN V4 (89.04%) vs MobileNetV2 (94.52%)

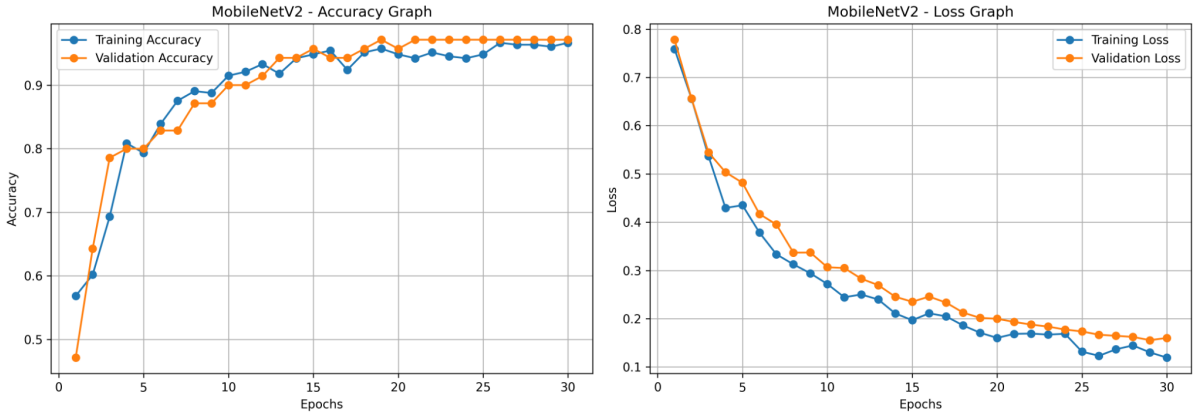




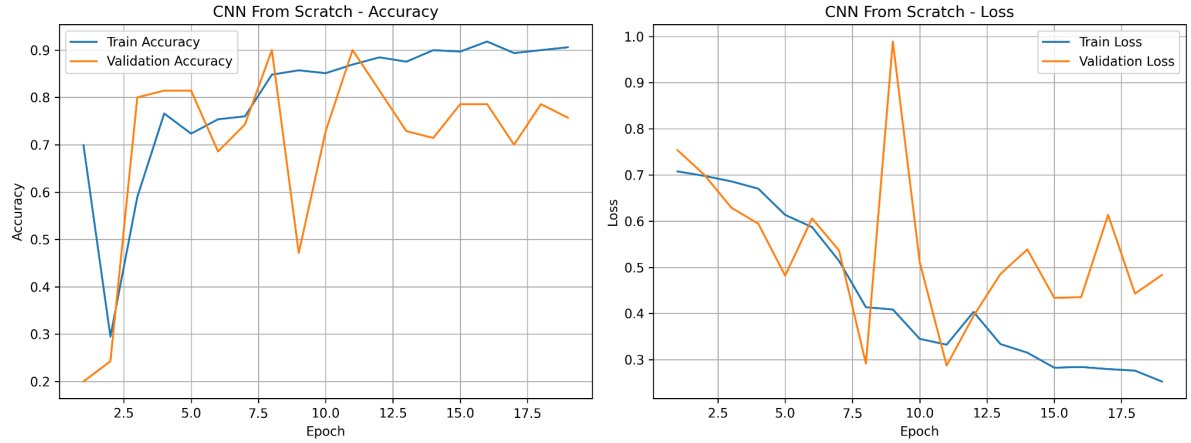
Custom CNN V4 confusion matrix



MobileNetV2 training curves



Custom CNN V4 training curves



5. Web Application and Deployment

The user interface is built with HTML, CSS, and JavaScript and deployed on Vercel. A FastAPI backend deployed on Render exposes GET /health and POST /predict. The prediction endpoint accepts JPG, PNG, or WEBP images and a selected model (mobilenet or scratch), resizes inputs to 224 × 224 RGB, and returns a predicted class, class probabilities, confidence, and a responsible-use disclaimer. Model weights are distributed separately through a GitHub Release.

6. Limitations and Responsible Use

The test set includes only 73 images, of which 16 are Spoiled. Exact-duplicate removal does not exclude visually similar images across splits. Performance may change with unseen cameras, lighting, bread varieties, or early-stage spoilage. Visible-image classification cannot establish microbiological safety or detect invisible contamination. A Fresh result must never be interpreted as a guarantee that bread is safe to consume.

7. Conclusion

MobileNetV2 achieved higher test accuracy (94.52%) than the custom CNN V4 (89.04%) on the project test set, with higher spoiled-class recall (81.25% versus 75.00%). The deployed prototype demonstrates an end-to-end visual-screening workflow. Further work should focus on larger, properly licensed datasets, external validation, calibration, and improved recall for spoiled examples.

8. Project Resources

Repository: https://github.com/Suf-tech/BreadGuard-AI

Live frontend: https://breadguard-ai-frontend.vercel.app/

Backend health: https://breadguard-ai-api.onrender.com/health

Model ZIP: https://github.com/Suf-tech/BreadGuard-AI/releases/download/v1.0.0/breadguard_trained_models.zip

9. Evidence and Citation Note

This report is based on the project README and supplied result plots. The original image-source list, complete training configuration, and external literature bibliography were not provided; therefore, no unverified source references or hyperparameters are invented. Dataset licensing and image attribution should be documented before public redistribution.